

The chunk-reserve protocol

Allocating one-time pads among m parties who all broadcast, never take turns, and each see a picture of the world that is up to d messages out of date — stranding $d(2m-1) - m$ pads in the worst case, a figure that does not grow with the number of pads.

Protocol chunk-reserve **Waste** $O(m \cdot d)$, independent of n **Verification** 310 tests, 180-cell grid

36 worst-case pads wasted at $m=5, d=5$ — down from 4000	0 times a party was ever blocked waiting on the network	exact the waste bound is reached, not merely respected	2 defects found in the handout's own two-party protocol
---	---	--	---

01 The problem

m parties share n random L -bit one-time pads $k_1 \dots k_n$. Every message is broadcast: the network hands it to all the other $m-1$ parties at the same instant. Different messages take different amounts of time, and at any moment at most d messages are undelivered. Each party sends whenever it wants — nobody takes turns, and nobody negotiates first.

Two things have to be right at once.

Perfect secrecy. No pad may ever be used twice, by the same party or by two different ones. A repeat destroys the one-time-pad guarantee outright: XOR the two ciphertexts and the pad drops out, leaving $m_1 \oplus m_2$ — a classic two-time-pad break, and from there natural language redundancy finishes the job. This is a safety property, so it must hold under *every* schedule and *every* delivery order, not merely the likely ones.

Few wasted pads. When nobody can safely send any more, the number of never-used pads should be small. The handout's two-party protocol wastes d . The target is something comparably tight — and in particular something that does *not* scale with n .

The tension between them is the whole problem. A party must decide *locally* which pad to use, while its picture of what everyone else has already used is up to d messages out of date. Play it safe and you waste pads; play it tight and two parties eventually reach for the same one.

02 What was there before

The starting point was a static partition: party j gets the fixed slice $[j \cdot n/m, (j+1) \cdot n/m)$ and never leaves it. It is trivially secret, and it is the obvious first answer. It has three problems, and they are worth stating because each one shaped the design that replaced it.

It wastes a constant fraction of everything. A party can only ever spend its own slice, so if one party does most of the talking the other $(m-1)/m$ of the pads are never touched. At $m = 5$ and $n = 5\,000$ that is **4 000 wasted pads**. Waste scales with n , which is precisely the failure mode the problem asks you to avoid.

It never modelled the delay. The original simulator kept one global array of pointers and mutated it directly on send, so every party saw every send instantly. The d bound was recorded but never bit. A protocol whose safety argument depends on a stale view was being tested in a world where no view is ever stale — the check could not have caught a violation even if one existed.

Its gap check was copied from the handout, including the bug. More on that in §7 — it turns out the handout's own guard admits a real collision.

03 Three dead ends

Before the design that worked, three that do not — each fails for a different reason, and the reasons are what point at the answer.

FIXED SLICES

Trivially secret, needs no coordination, and useless: a quiet party's slice is dead capital. Waste is $\theta(n)$, not $\theta(d)$.

M POINTERS IN A LINE

Generalise the two-party protocol by running m pointers up the pad line, each keeping a d gap from its neighbour. Only the top pointer has anywhere to go; everyone else jams against the party in front immediately.

"LOWEST UNUSED PAD"

Two parties pick the same pad whenever both messages are in flight. Breaking the tie with a per-party offset only works if the offset is a fixed residue class — which is the static partition again.

The third one generalises into a real obstruction: **you cannot break ties with a rule that is both local and static.** Any deterministic function of a party's own identity and the shared pad list either collides (two parties map to the

same pad) or partitions (each party is confined to its own residue class). To escape, the tie-break has to depend on something that changes over time and that every party agrees on.

04 The opening

That something is handed over for free in the problem statement, and it is easy to read past:

“the network delivers each message at the same time to all $m-1$ parties.”

Simultaneous delivery means every party observes the *same delivered sequence, in the same order*. Parties disagree about the last d messages — those are still in flight, and that disagreement is irreducible — but they agree *perfectly* about everything before that. The delivered history is **common knowledge**.

So any allocation rule that is a function of the delivered history alone gives every party the same answer, with no negotiation and no extra round trips. The entire design problem reduces to a single question: *find a rule over the delivered history that hands out pads fast enough that nobody ever has to wait.*

05 Chunk-reserve

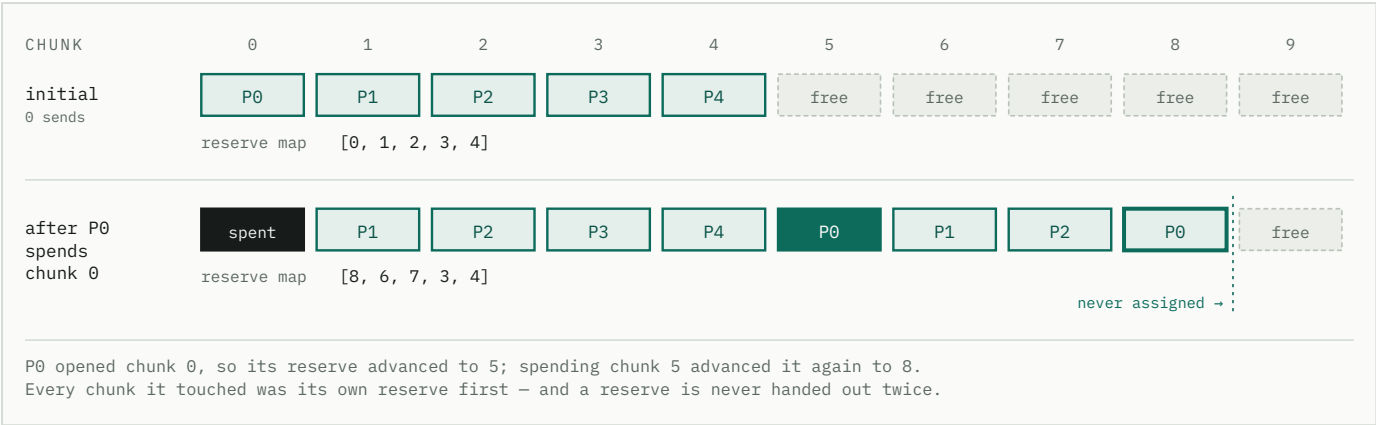
Group the pads into chunks of $c = d$ consecutive pads. Every party holds two chunks at a time:

- a **current** chunk it is drawing pads from — private state, nobody else knows how far along it is;
- a **reserve** chunk — public, and identical in everyone's book.

Initially party j has $\text{current} = \text{reserve} = \text{chunk } j$. Messages carry exactly what they carried in the handout's two-party protocol: the ciphertext and the pad index. The whole coordination rule is one line, replayed by every party over the delivered sequence:

When a delivered message from party j uses a pad inside $\text{reserve}[j]$, set $\text{reserve}[j]$ to the next chunk that has never been assigned to anyone.

A party spends its current chunk in order; when it runs out, it moves into its reserve. That is the entire protocol. No requests, no grants, no acknowledgements, no extra round trips, and no negotiation of any kind.



 held as reserve  current — being spent  fully spent  never assigned

Figure 1 — the pad line, before and after. Ten chunks of $c = d = 3$ pads. Colour encodes allocation *state*; the label carries party identity. Chunks are handed out strictly left to right and a chunk, once assigned, is never reassigned — so the reserve map is always a partition, and that is the whole secrecy argument.

The rule running

This is real output from `main.py` at $n = 5\,000, d = 3, m = 5$. Watch the reserve map: it changes on *deliveries*, never on sends, which is what makes it common knowledge.

event	reserve map	pads used
initial	[0, 1, 2, 3, 4]	–
party 0 sends on pad 1	[0, 1, 2, 3, 4]	1
party 0 sends on pad 2	[0, 1, 2, 3, 4]	2
party 1 sends on pad 4	[0, 1, 2, 3, 4]	3
network delivers party 0's pad 1	[5, 1, 2, 3, 4]	3 ← P0 gets chunk 5
party 0 sends on pad 3	[5, 1, 2, 3, 4]	4
network delivers party 0's pad 2	[5, 1, 2, 3, 4]	4 ← no change: chunk 0 is already open
party 2 sends on pad 7	[5, 1, 2, 3, 4]	5
network delivers party 1's pad 4	[5, 6, 2, 3, 4]	5 ← P1 gets chunk 6
party 0 sends on pad 16	[5, 6, 2, 3, 4]	6 ← P0 has moved into chunk 5
network delivers party 2's pad 7	[5, 6, 7, 3, 4]	7
party 1 sends on pad 5	[5, 6, 7, 3, 4]	8
network delivers party 0's pad 16	[8, 6, 7, 3, 4]	8 ← P0 gets chunk 8

Two details in that trace carry the design. First, the second delivery from party 0 changes nothing — the rule only fires on a pad inside the party's *current reserve*, so it is idempotent against stragglers from a chunk already opened. Second, party 0 sends on pad 16 before anyone has acknowledged anything; it moved into chunk 5 the moment the delivery told it the chunk was its, and the delivery told all four other parties the same thing at the same instant.

06 Analysis

6.1 Perfect secrecy

THEOREM 1

No pad is ever used twice, under any send schedule and any delivery order.

PROOF

Consider the reserve map as a function of the delivered sequence. The initial reserves $0 \dots m-1$ are pairwise distinct. Each reassignment takes `next_chunk`, a counter that only ever increments and is never revisited, so no chunk is assigned to two parties, and no chunk is assigned to the same party twice at different times.

A party draws pads only from its current chunk, and its current chunk is only ever set to a chunk that was its own reserve. Chunks tile the pads $1 \dots n$ disjointly, and within a chunk a party advances a private cursor that only increments. Therefore every pad is drawn by at most one party, at most once. ■

Note what that argument does *not* use: no assumption about timing, about delivery order, about how many messages are in flight, or about how stale any party's view is. All of those affect how *fast* reserves advance; none of them can make two parties hold the same chunk. **Staleness costs throughput, never secrecy.** That separation is the reason the protocol is worth trusting: the safety property does not depend on the network behaving.

6.2 Wait-freedom, and why $c = d$ exactly

THEOREM 2

A party the network will accept a message from can always produce a pad, provided unassigned chunks remain. It never has to wait on a delivery.

PROOF

Suppose the network will accept another message from party j . Then j has at most $d-1$ of its own messages undelivered.

If j 's current chunk still has an unused pad, it sends. Otherwise it has already sent all $c = d$ pads of that chunk. At most $d-1$ of those are still in flight, so at least $c - (d-1) = 1$ has been delivered — and the first such delivery fired the rule and advanced j 's reserve to a fresh chunk. So j has a reserve it has not yet opened, and moves into it. ■

The bound is tight, and that is worth stating precisely because it is the one place where the constant matters. With $c = d - 1$ a party can put its entire chunk in flight and have nothing delivered, leaving it stuck with capacity on the network and no pad to use. Both directions are pinned by `test_chunk_size_d_is_wait_free_and_d_minus_one_is_not`: at $c = d$ the blocked counter stays at zero across the grid, and at $c = d-1$ a party strands.

The proof only ever uses “at most d undelivered”, so it holds under both readings of the handout's bound — d capping the total number of undelivered messages (the m -party wording) or capping each sender separately (the two-party wording). Both are simulated, and both pass.

6.3 Wasted pads

THEOREM 3 — EXACT

When the chunks run out, the number of never-used pads is *at most* $(m-1)(2d - 1) + (d - 1) + (n \bmod d)$ — and some schedule achieves that figure exactly.

PROOF

Upper bound. At the moment the last chunk is assigned, a party holds at most two chunks: one reserve it has not opened (at most c unused pads) and one current chunk it is partway through (at most $c - 1$ unused, since opening it consumed a pad). So it strands at most $2c - 1$.

One party is special. Let T be the party whose delivery exhausted the supply — the delivery that found no chunk left and set `reserve[T] = None`. T therefore holds *no* reserve afterwards, only the chunk it is spending, so it strands at most $c - 1$. Someone must exhaust the supply, so at most $m - 1$ parties strand $2c - 1$, giving $(m-1)(2c - 1) + (c - 1)$. The oversized final chunk contributes its remainder $n \bmod c$ wherever it lands.

Achievability. Let each of parties $1..m-1$ send exactly one message and then fall silent. That send opens its initial chunk, stranding $c - 1$, and its delivery advances its reserve to a fresh chunk it never opens, stranding c more — $2c - 1$ each. Party 0 then drains the supply and stops the instant its own reserve goes `None`, stranding the $c - 1$ pads left in the chunk it holds. The total is the bound. ■

$$\text{waste} = d(2m - 1) - m + (n \bmod d)$$

There is no n in it. Doubling the number of pads does not cost a single additional stranded pad — exactly the property the static partition lacks, and the reason the problem is interesting at all.

6.4 Where the worst case actually lives

The achievability half of that proof is the surprising part, and it is worth isolating because it is not the case intuition offers.

A party that *never speaks* strands one chunk — d pads. A party that *keeps speaking* strands nothing: when the supply runs out its reserve goes `None` and it simply drains what it holds. Neither extreme is bad. The damage peaks strictly in between:

A party that sends exactly one message and then falls silent strands $2d - 1$ — the $d-1$ pads left in the chunk that one send opened, plus the entire fresh reserve the delivery earned it.

One message is the worst thing a party can do. It is enough to claim a second chunk and not nearly enough to use either of them.

CORRECTION TO THE EARLIER EVALUATION

The original grid understated the worst case by nearly half

None of the five original schedules — `single`, `round_robin`, `uniform`, `skewed`, `bursty` — ever produces a party that speaks once and stops. All five agreed on 20 wasted pads at $m = 5$, $d = 5$. The true figure is 36.

The `one_shot` schedule is that adversary and is now part of the grid. The lesson generalises past this protocol: a schedule space that only varies *how much* parties talk will miss a worst case that depends on *when they stop*.

6.5 Optimality when parties are quiet

One case deserves separating out, because there the protocol is not merely good but provably optimal. When $m-1$ parties stay silent *from the start* — never speaking at all, rather than speaking once — waste is exactly $(m-1) \cdot d$, one chunk per silent party.

LOWER BOUND

Any protocol in which a party can send d messages without first asking anyone must leave $(m-1) \cdot d$ pads unusable when $m-1$ parties are quiet.

This is the handout's own two-party lower bound, applied once per silent party. If a party must be able to speak d times with no round trip, then at every moment it is holding d pads that no other party may touch — otherwise some other party could consume one of them while the first party's messages are in flight, and that is a collision. With $m-1$ parties quiet, that is $(m-1) \cdot d$ pads permanently out of reach. ■

Chunk-reserve hits that figure exactly, at every d tested and for both $m = 5$ and $m = 9$. So in the case that sinks the static partition — one party doing all the talking — the protocol is optimal, not merely better. The one_shot worst case of §6.4 is a different scenario, and no matching lower bound is claimed for it.

07 Two defects in the handout's protocol

The two-party protocol in the problem statement was implemented as a reference point, to check the m -party protocol degenerates to something sensible and to have a baseline for waste. Running it against the adversarial delivery policy turned up two genuine defects.

FINDING 1 — OFF-BY-ONE

The gap check admits a real collision

The handout guards with $i_2 - i_1 - 1 \geq d$. Alice knows Bob's pointer only as of the last delivery, and Bob may since have advanced a further d , so her pad i_1+1 is safe only when $i_1 + 1 < i_2 - d$ — that is, $i_2 - i_1 - 1 \geq d + 1$.

Smallest counterexample, with $n = 4$, $d = 1$: Bob uses pad 4 (delivered), then pad 3 (still in flight). Alice still sees $i_2 = 4$, so the handout's check passes all the way to $i_1 = 2$ and she takes pad 3 — the pad Bob is using right now. Two ciphertexts, one pad.

FINDING 2 — UNSOUND VIEW UPDATE

A pointer view has to be folded monotonically

The handout bounds how many messages are outstanding, not how long any one of them takes, so deliveries can arrive out of order. Taking the newest arrival at face value — `seen = msg.pointer` — lets Alice's view of Bob's pointer move *backwards*, which reopens pads Bob has already claimed.

Under LIFO delivery this produces a real collision. Each pointer only ever moves one way, so each party must fold its observations with `min / max`. This one was found by the adversarial delivery policy, not by reading the code — which is the argument for building the hostile network before trusting the protocol.

Both are pinned by tests. `TwoPartyPointerProtocol(..., handout_guard=True)` reproduces the handout's guard verbatim so the collision can be exhibited as an assertion rather than asserted in prose; the corrected guard wastes `d + 1` and is collision-free under every delivery order tested.

08 The model question

Chunk-reserve needs the delivered history to be common knowledge, and it needs one thing more that is worth being explicit about: **a party's reserve advances when its own message lands, so it has to observe that event**. That is the ordinary semantics of a broadcast medium — an atomic or total-order broadcast delivers to every participant, sender included — and it is what the problem statement describes.

If a sender gets *no* feedback about its own messages, the rule is not computable: the new reserve depends on how many other parties opened chunks first, and a sender cannot place its own deliveries in that order. Chunk-reserve stays *safe* in that weaker model — Theorem 1 uses no timing assumption — but it stalls almost immediately. That is tested rather than assumed (`test_without_delivery_feedback_the_ledger_splits_but_stays_safe`).

So a second protocol was built for the weaker model, to show the problem is still solvable there and at what price. **GrantProtocol** uses static chunk ownership plus explicit hand-offs: chunk *C* starts owned by party *C mod m*; a party running low broadcasts a request; a party with spare untouched chunks gives half of them away and drops them from its own pool *at send time*, before the grant is even in the air.

Safety there needs no timing argument at all — a chunk is only ever used by its owner, and ownership moves only when the owner unilaterally gives it up, so the window during which a grant is in flight belongs to nobody. The cost is liveness: rebalancing takes a round trip, so parties stall waiting for grants. It matches chunk-reserve on waste and pays for it in the *blocked* column below.

09 Evaluation

The simulator's job is to be *unkind*. Each of the 180 grid cells is the worst case over 4 delivery orders × 2 network pressures × 3 seeds, and every pad handed out is checked against every pad handed out before it.

WASTED PADS · M = 5, D = 5, N = 5 000, BY SEND SCHEDULE

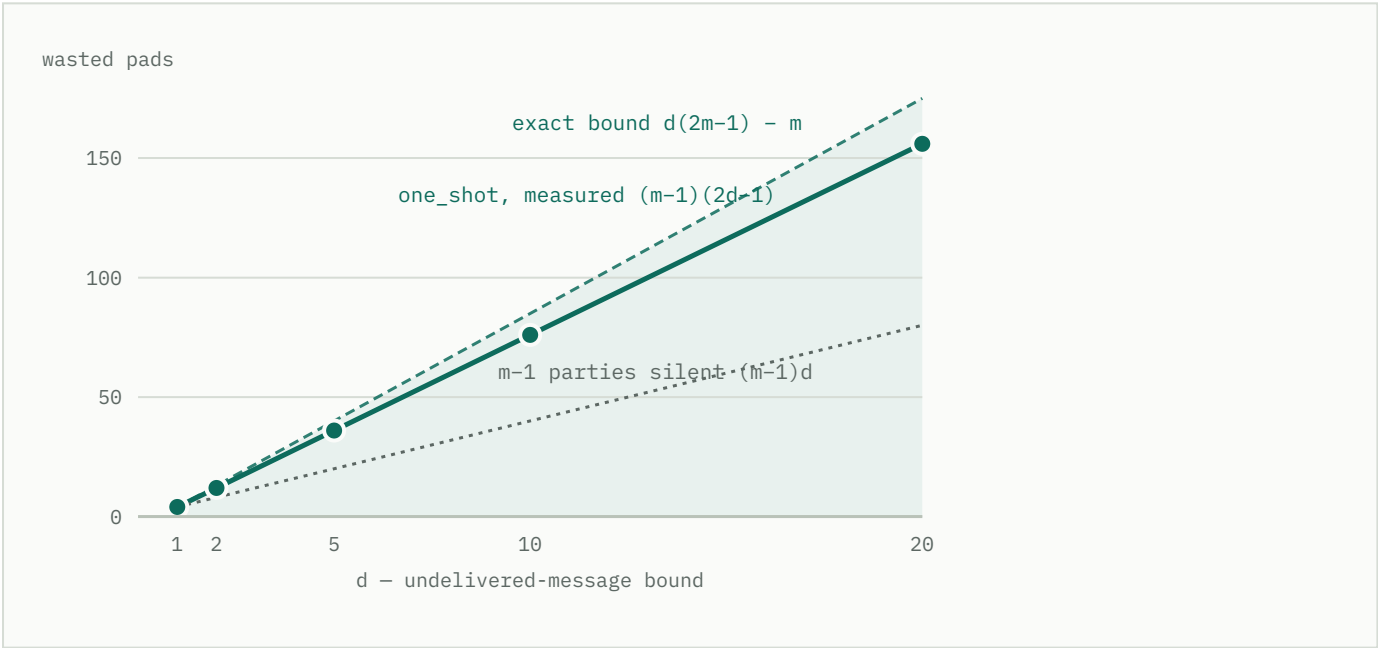
Protocol	one party talks	mostly one	one_shot	everyone talks
chunk-reserve	20	0	36	0
grant	20	0	56	0
static partition	4 000	0	3 996	0

`one_shot` is the column that separates chunk-reserve from `grant`. Under the five original schedules the two were indistinguishable on waste at every *d*; stress them properly and `grant` strands roughly twice as much.

Axis	Values	What it stresses
schedule	single, round_robin, uniform, skewed, bursty, one_shot	who talks, how unevenly — and, in one_shot, when they stop
delivery	random, fifo, lifo, adversarial	which undelivered message the network releases next
pressure	eager, lazy	lazy delivers nothing until a party is starved, pinning the backlog at d all run
bound_mode	global, per_party	both readings of the handout's d bound

The `adversarial` policy deliberately holds back each party's *oldest* message — in chunk-reserve that is the one that would advance the sender's reserve, and in the pointer protocol the one that moves the pointer the other party is watching. Paired with `lazy` pressure it is the worst case the protocol has to survive, and it is what the headline figures use.

9.1 Waste against the bound



one_shot — measured worst case exact bound, Theorem 3 m-1 parties silent — the mild case

Figure 2 — three different questions, three different lines. Worst case over every schedule, delivery order, pressure and seed at $n = 5\,000$. The lower dotted line is the mild case that used to be reported as the worst: $m-1$ parties silent from the start. The solid line is what the `one_shot` adversary actually reaches. The dashed ceiling is the exact bound of Theorem 3 — the remaining $d-1$ needs the draining party to stop mid-chunk as well, which no schedule can express, so a unit test drives it directly. At $m = 9$ the shape is identical against $(m-1)(2d-1) = 8(2d-1)$.

WORST CASE OVER EVERY SCHEDULE, DELIVERY ORDER AND PRESSURE · N = 5 000, M = 5

d	chunk-reserve	exact bound	grant	static partition	old grid said
1	4	4	4	4 000	4
2	12	13	20	4 000	8
5	36	40	56	4 000	20
10	76	85	146	4 000	40
20	156	175	316	4 000	80

At $m = 9$ the same pattern holds against $(m-1)d = 8d$: 8 wasted at $d = 1$, 40 at $d = 5$, 160 at $d = 20$ — each exactly the optimum — against 4 444 for the static partition at every d .

9.2 Liveness: who stalls

Blocked counts the times a party wanted to send, the network had room, unused pads existed, and the protocol still could not give it one. It is the operational reading of wait-freedom, and it is where the three protocols actually separate most sharply.

TIMES BLOCKED · M = 5, D = 5, N = 5 000, BY SEND SCHEDULE

Protocol	single	round robin	uniform	skewed	bursty	one_shot
chunk-reserve	0	0	0	0	0	0
grant	16	44	124	625	196	66
static partition	8	24	344	22 042	834	7

Zero, everywhere, is Theorem 2 showing up as a measurement. The other two are not broken — they are safe — they just make parties wait, which is the cost the problem statement explicitly asks you to avoid when it rules out “the impractical solution where the parties continuously agree on who is the next sender”.

9.3 Waste does not grow with n

The headline property, stated as a measurement — and now stated about the *worst* case rather than a mild one. Same protocol, same $d = 5$, same `one_shot` adversary, n increased 64-fold:

CHUNK-RESERVE · M = 5, D = 5, ONE_SHOT WORST CASE

n	wasted pads	messages sent	utilisation
5 000	36	4 984	99.680 %
20 000	36	19 984	99.920 %
80 000	36	79 984	99.980 %
320 000	36	319 984	99.995 %

Thirty-six, four times over. The static partition on the same runs wastes 4 000, 16 000, 64 000 and 256 000 — it is $0.8n$ by construction. Throughput is about $3\text{ }\mu\text{s}$ per message, dominated by the simulator rather than the protocol; the protocol itself is a couple of integer comparisons per send.

10 Verification

Secrecy is a property you cannot establish by running the happy path, so the check is built into the driver rather than bolted on: `simulation.run` records every pad handed out and raises `PadReuseError` the moment one repeats. **Any run that completes is therefore a proof of no reuse for that schedule**, and the test suite's job reduces to running as many different schedules as possible.

310 tests, about eight seconds. Each protocol is driven under $6\text{ schedules} \times 4\text{ delivery orders} \times 2\text{ pressures} \times$ both readings of the d bound, for $m \in \{5, 9\}$ and d from 1 to 17, several seeds each.

The obvious objection to “every completed run proves no reuse” is that the check might never fire — a suite that passes vacuously. `test_pad_reuse_is_actually_detected` answers it by running a deliberately broken protocol and asserting the check catches it.

Test	Property
no_pad_is_used_twice	secrecy across the full schedule × delivery × pressure grid
secrecy_holds_for_both_readings_of_d	the d bound read globally and per party
reserves_stay_exclusive_and_are_never_reissued	the reserve map really is a partition — Theorem 1 checked directly, not inferred from the absence of collisions
all_parties_derive_the_same_reserve_map...	common knowledge actually holds in the broadcast model
chunk_size_d_is_wait_free_and_d_minus_one_is_not	$c = d$ is the exact threshold — both sides of it
never_blocked_under_any_schedule	Theorem 2, with the network held at capacity
waste_never_exceeds_the_proved_bound	Theorem 3 across the grid
the_waste_bound_is_tight	the bound is <i>reached</i> — it is the worst case, not a ceiling — with and without a remainder
one_shot_is_the_worst_schedule_in_the_grid	speaking once and stopping strands more than never speaking, or never stopping
waste_does_not_grow_with_n	same waste at $n = 2\,000, 20\,000, 200\,000$
beats_the_static_partition_when_one_party...	50× less waste than the baseline
the_handout_guard_admits_a_collision...	Finding 1, as a concrete collision
two_party_reference_is_safe_and_wastes_at_most_d	the corrected two-party protocol
without_delivery_feedback_the_ledger_splits...	chunk-reserve stalls without the broadcast model, but never leaks
grant_protocol_is_safe_without_delivery_feedback	the fallback is safe in the weaker model
grant_protocol_rebalances_but_has_to_wait_for_it	the fallback's liveness cost is real
runs_are_reproducible	same seed, same run
pad_reuse_is_actually_detected	the check itself fires — the suite is not vacuous

11 Implementation

1 567 lines of Python across five modules, no dependencies beyond `pytest`. Everything is pure `stdlib`; the pads themselves are never materialised, since the protocol is about *which index* to use and the XOR is not the interesting part.

File	lines	Contents
protocol.py	547	ChunkReserveProtocol, plus GrantProtocol, StaticPartitionProtocol and TwoPartyPointerProtocol behind one interface
test_protocol.py	397	310 tests
simulation.py	325	network with the d bound, four delivery adversaries, five send schedules, the driver, the pad-reuse check
eval_protocol.py	207	the 180-cell grid; writes summary.csv
main.py	91	the annotated trace and the head-to-head comparison

The separation that made the rest possible is between `protocol.py`, which knows nothing about time, and `simulation.py`, which owns all of it. A protocol exposes `can_send`, `send` and `observe`; the simulator decides who talks and what the network does to them. That is why four different protocols can be driven interchangeably, and why the adversarial delivery policy could be pointed at the handout's two-party protocol and find a bug in it.

11.1 Repository engineering

Work beyond the protocol itself, since it affects whether the results can be trusted or reproduced.

Continuous integration. The suite runs on Python 3.11, 3.12 and 3.13, and the demo and a small evaluation grid are smoke-run so neither entry point can rot unnoticed. A lint job runs `ruff check` and `ruff format --check` against a config recorded in `pyproject.toml`.

summary.csv is reproducible. The results file originally carried a wall-clock `us_per_send` column, so re-running the evaluation rewrote every row every time and left the working tree dirty for no real change. Every other column is a deterministic function of `(protocol, m, d, n, schedule)`. Moving the timing to stdout makes the file reproduce byte for byte — verified across repeated runs, across Python 3.13 and 3.14, and across macOS and Linux. CI now diffs the committed file against a fresh run, so a **protocol change that was never re-evaluated fails the build** and the published numbers cannot silently drift from the code.

One latent portability bug was caught by the matrix. An f-string used PEP 701 nested same-quote syntax, which parses only on Python 3.12+. It ran fine locally on 3.14 and would have failed for anyone on 3.11.

12 Limits and honest caveats

The optimality claim is scoped. $(m-1)d$ is proved optimal only for the case where $m-1$ parties are silent from the start. The worst case of §6.4, $d(2m-1) - m$, is now known *exactly* — it is reached, not merely respected — but that is a statement about this protocol, not about the problem. **No matching lower bound is offered for the one-shot case**, so whether any wait-free protocol must strand $\Theta(md)$ there, or whether a cleverer allocation rule does better, is still open. It is the obvious next question.

An earlier version of this report got the worst case wrong. It reported $(m-1)d$ — 20 pads at $m = 5$, $d = 5$ — as the measured worst case, because none of the five schedules in the grid at the time produced a party that speaks once and stops. The real figure is 36. The error was in the evaluation, not the protocol, and it is worth naming as a

caution: an exhaustive-looking sweep is only as good as the dimensions it varies, and “when does a party stop talking” was not one of them.

Everything is simulated, not distributed. The parties are objects in one process. The simulator enforces the `d` bound and is deliberately adversarial about ordering, but real networks also partition, duplicate and reorder in ways the model does not cover, and there is no authentication: a party that lies about which pad it used breaks the scheme immediately. The problem statement assumes honest parties, so that is in scope, but it is an assumption worth naming.

The broadcast requirement is real. Chunk-reserve needs senders to observe their own deliveries. §8 covers what happens otherwise and offers a fallback, but the headline results assume the model the problem statement describes.

Exhaustive testing is not proof. 310 tests across a large parameter grid is strong evidence, not a formal guarantee. The proofs in §6 are the actual argument; the tests exist to catch the gap between the proof and the code. A model-checked or mechanised version of Theorem 1 would close that gap.

NYU CS6903 / 4783 — Project 1. Protocol, analysis, simulator and evaluation as described above; all figures are generated from the committed `summary.csv`, which CI verifies against a fresh run of `eval_protocol.py`.